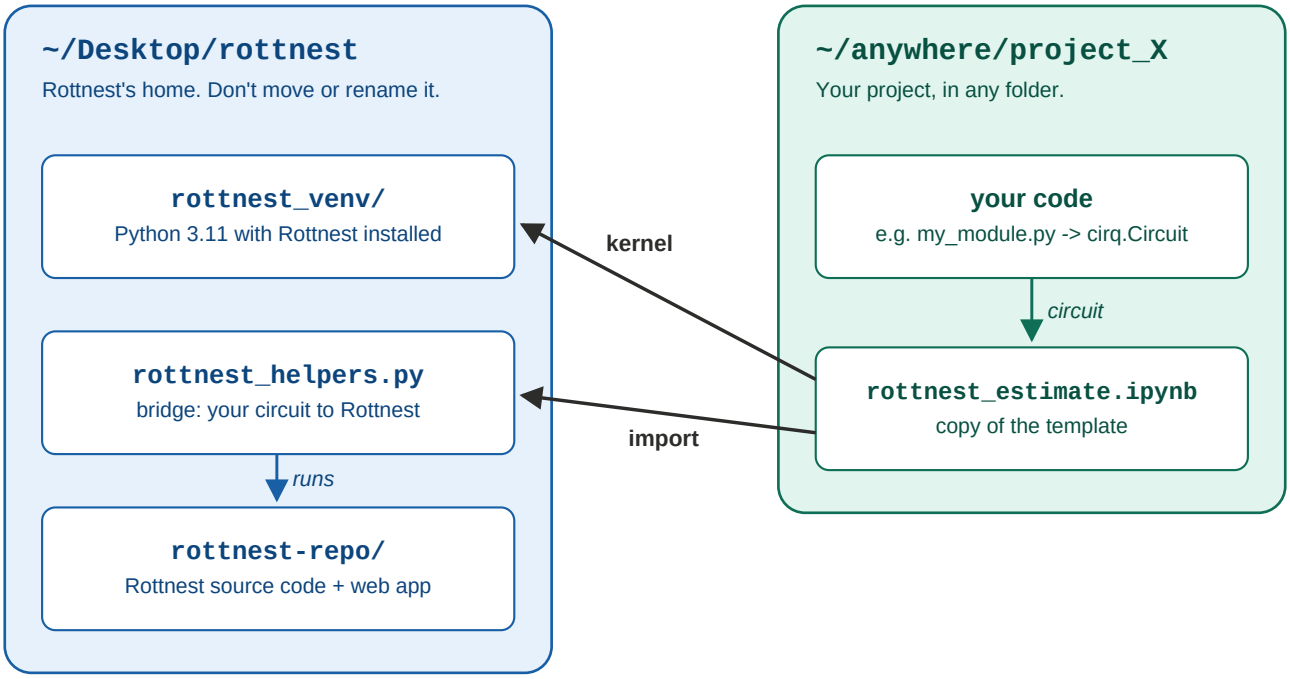
Using Rottnest from any project

A quick-start guide for your Mac setup: get quantum resource estimates (rotations, T gates, runtime, chip volume) for any Cirq circuit.

The idea in one picture



Rottnest lives in one fixed place. Your projects can live **anywhere**. Three links make it work:

Link	What it means	Why it works from any folder
kernel	The notebook runs on Rottnest's Python: `rottnest_venv`.	Rottnest is installed inside that Python environment, not inside a folder.
import	The notebook imports `rottnest_helpers`.	Its folder is registered in `rottnest_venv`, so Python finds it from anywhere.
circuit	Your own code builds the circuit.	The notebook sits in your project folder, so it can import your modules directly.

> **Why a helper file at all?** Rottnest runs each circuit in separate worker processes, and those processes can't see variables in your notebook. `rottnest_helpers.py` saves your circuit to a temporary file the workers can read, runs Rottnest, and hands the results back.

Start a new project in 5 steps

Do this once per project. Nothing needs installing again.

1 Open your project folder in VS Code

File > Open Folder... and choose your project (for example `~/Documents/project_X`). It can be anywhere.

2 Copy the template notebook into it

Copy this file into your project folder and rename it, for example to `rottnest_estimate.ipynb`:

```
~/Desktop/rottnest/rottnest_estimate_template.ipynb
```

Or, in VS Code's terminal (**Terminal > New Terminal**):

```
cp ~/Desktop/rottnest/rottnest_estimate_template.ipynb rottnest_estimate.ipynb
```

3 Choose Rottnest's Python as the kernel

Open the notebook and click **Select Kernel** (top right) > **Python Environments** > `rottnest_venv`.

Not in the list? Choose **Select Another Kernel > Python Environments > Enter interpreter path** and paste:

```
~/Desktop/rottnest/rottnest_venv/bin/python
```

4 Put in your project's circuit

In notebook section 2, replace the example circuit with a call to your code:

```
from my_module import build_circuit
circuit = build_circuit(...)
```

If your code needs extra packages, install them into the same Python:

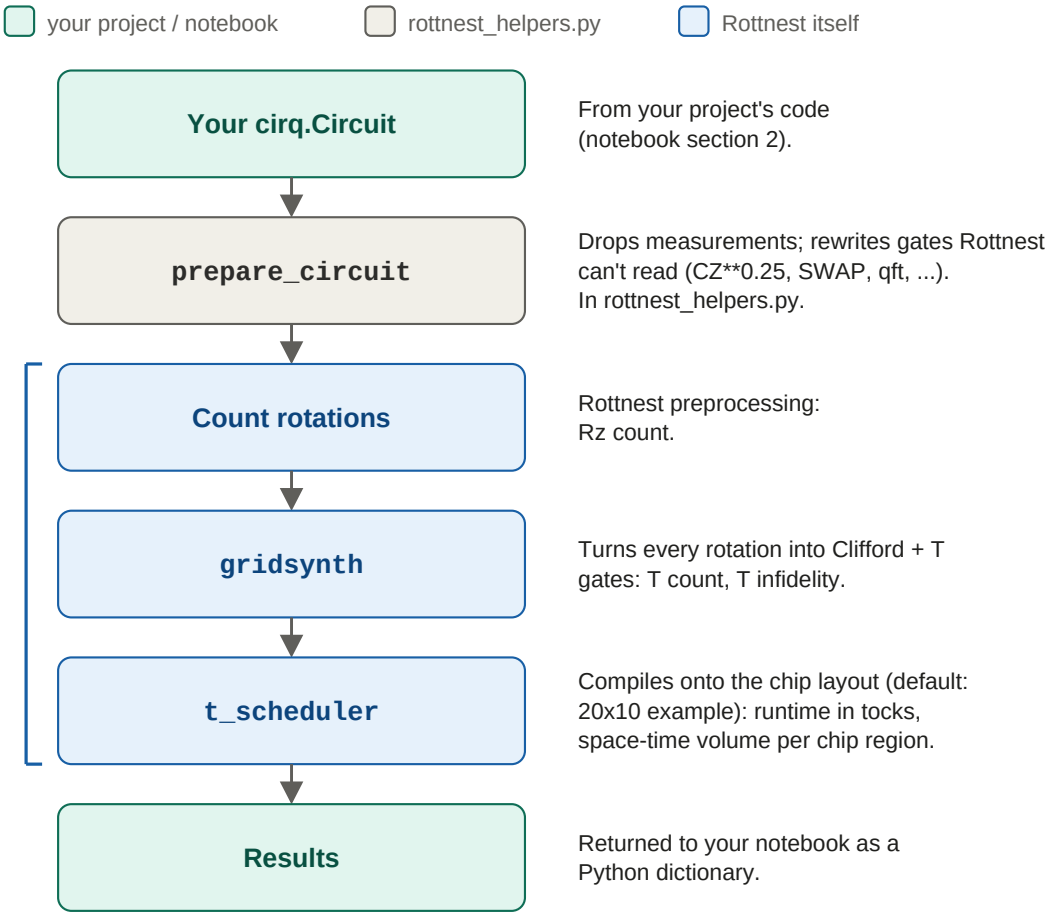
```
~/Desktop/rottnest/rottnest_venv/bin/pip install PACKAGE
```

5 Run All

Click **Run All**. Small circuits finish in a few seconds. You get the headline numbers, a table and chart of every parameter Rottnest reports, and charts of how each parameter grows with qubit count. The charts, the numbers and the full result are saved in your project folder.

> **Only need the numbers?** Any Python file or notebook using the `rottnest_venv` kernel can do it in two lines:
> `from rottnest_helpers import count_cirq_circuit`
> `result = count_cirq_circuit(circuit)`

What happens when you run it

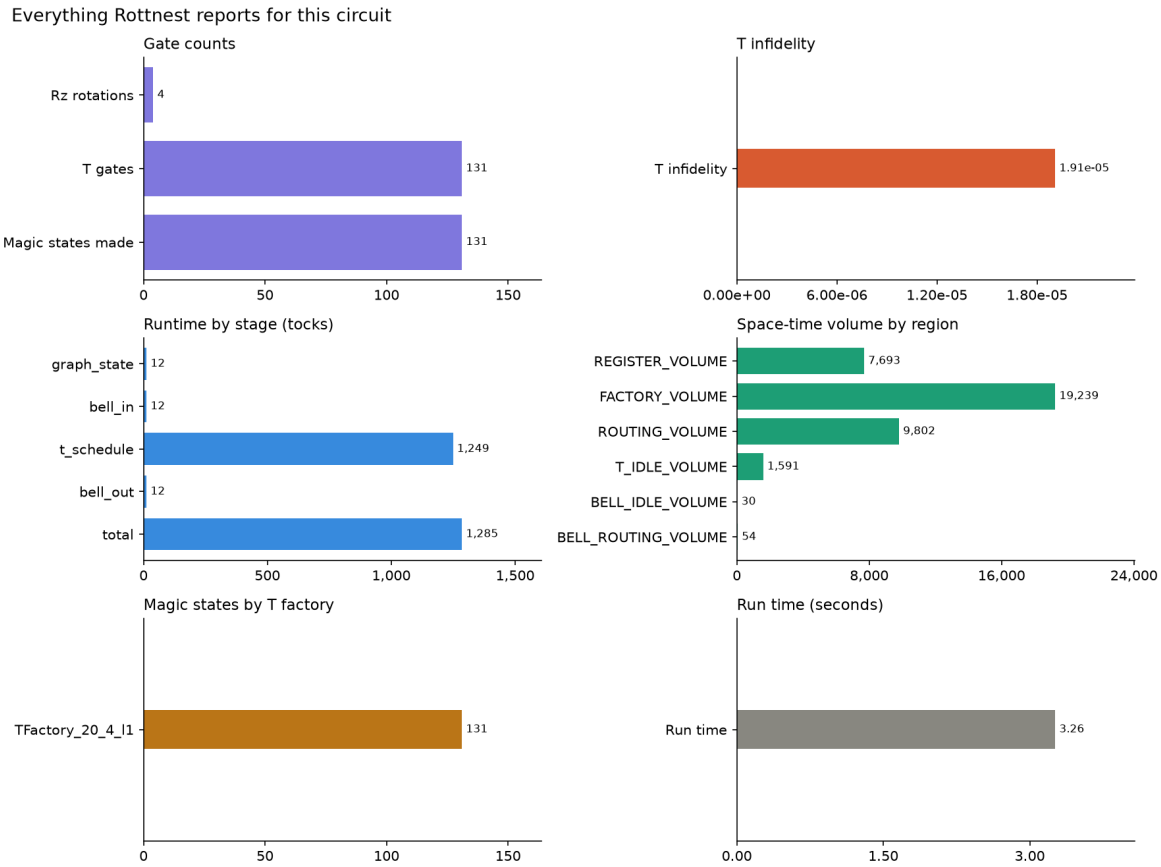


The template notebook, section by section

Section	What it does	Why
1. Check the setup	Prints which Python and helper file are in use.	Both paths must point into `~/Desktop/rottnest`. If not, the kernel is wrong.
2. Get the circuit	Builds or imports your cirq.Circuit.	You change this, and `circuit_for(n)` in section 6, for each project.
3. Prepare	Shows what Rottnest will receive and checks it is the same operation.	Rottnest silently miscounts unsupported gates. The check is skipped above 10 qubits.
4. Run Rottnest	Runs the full estimate and shows the headline numbers.	Options: `layout=`, `verbose=True`, `timeout=`.
5. Every parameter	Table of every number Rottnest returns, plus a six-panel chart (saved as `rottnest_parameters.png`).	Shows where the cost goes. New fields from Rottnest appear automatically.
6. Scaling	Re-runs Rottnest for several sizes of your circuit (`circuit_for(n)`) and plots every parameter against qubit count.	Shows how the cost grows. Saves `rottnest_scaling.csv` and `rottnest_scaling.png`.
7. Save	Writes `rottnest_result.json`.	Keeps results with your project.

Reading the results

Result	Meaning
Rz rotations	Rotation gates in the circuit Rottnest received.
T gates	T gates after gridsynth turns every rotation into Clifford + T.
T infidelity	Estimated error from approximating the rotations.
Runtime (tocks)	Length of the compiled schedule in clock cycles, total and per stage.
Space-time volume	Chip area x time used by each region (registers, T factories, routing, ...).
Magic states made	T states produced by each T factory.
Run time	How long the estimate took on your Mac.



Real output of section 5 for the template's 3-qubit example circuit (also saved as `rottnest_parameters.png`). Your circuit's chart looks like this.

When something goes wrong

Symptom	Fix
`No module named rottnest_helpers` (or cirq)	Wrong kernel. Select `rottnest_venv` (step 3).
Your own module isn't found	Keep the notebook inside your project folder, next to your code.
Terminal says **Operation not permitted**	System Settings > Privacy & Security > Files & Folders: allow Desktop Folder for Terminal and for Visual Studio Code.
Run never finishes	Big circuits take longer. Use `timeout=` and `verbose=True`. Never choose the **Rz Counter** architecture (Rottnest bug).
Everything stopped working	Check that `~/Desktop/rottnest` wasn't moved or renamed.
`code: command not found`	Use `open -a "Visual Studio Code" .` instead.

Cheat sheet

Everything you need to remember, in one place.

What	Address or command
Rottnest's Python (the kernel)	`~/Desktop/rottnest/rottnest_venv/bin/python`
Template to copy into projects	`~/Desktop/rottnest/rottnest_estimate_template.ipynb`
Full tutorial with examples	`~/Desktop/rottnest/rottnest_tutorial.ipynb`
The helper (don't edit)	`~/Desktop/rottnest/rottnest_helpers.py`
Count a circuit in Python	`from rottnest_helpers import count_cirq_circuit` `result = count_cirq_circuit(circuit)`
Run a built-in example	`run_rottnest("Heisenberg", {"height": 2, "width": 2})`
Install a package for your project	`~/Desktop/rottnest/rottnest_venv/bin/pip install NAME`
Set up a Terminal window	`source ~/Desktop/rottnest/rottnest-env.sh`
Start the Rottnest web app	`source ~/Desktop/rottnest/rottnest-env.sh` `cd ~/Desktop/rottnest/rottnest-repo && ./run_rottnest.sh`
Stop the web app	Press Control + C in that Terminal.

Good to know

• Every call starts a fresh Rottnest run, so even tiny circuits take about 3 seconds.

• T counts can differ slightly between runs of the same circuit.

• Hardware numbers depend on the chip layout. The default is t_scheduler's 20x10 example; pass `layout=` to use another one. You can design layouts in the web app.

• Measurements are removed before counting; they don't change the resource estimate.

• Large circuits (hundreds of thousands of gates) haven't been tested yet. Start small.